به نام خدا

تمرین کامپیوتری سوم درس طراحی کامپایلر

زمستان ۱۴۰۴

طراحان: علی عطاءاللهی، مهدیس میرزایی

مقدمه

در این فاز بخش‌های مربوط به تولید کد را به کامپایلر خود اضافه می‌کنید. در انتهای این فاز، کامپایلر شما به طور کامل پیاده‌سازی شده و برنامه‌های نوشته شده به زبان را به کد قابل اجرا توسط ماشین تبدیل می‌کند. پیاده‌سازی شما باید به ازای هر فایل ورودی ,لیست bytecode های معادل آن را تولید کند. توجه کنید که شما برای تولید کد به اطلاعات جمع آوری شده در جدول علائم و اطلاعات مربوط به تایپ نودهای درخت AST نیاز دارید.

نکات کلی پیاده‌سازی

می‌دانیم در این زبان در هر لحظه ممکن است چندین رویداد رخ دهد و این همزمانی باید مدیریت شود. این مدیریت توسط Scheduler انجام می‌شود. به عبارتی هدف ما از استفاده از آن, مشخص کردن برقراری نظم در ایجاد bytecode ها است. به عنوان مثال اگر نمونه کد زیر را در نظر بگیریم باید در لحظات ۱۰ و ۲۰ و ۳۰ و ۴۰ به طور همزمان a و b و n را تغییر دهیم, این مدیریت و ترتیب اجرا را با استفاده از Scheduler به دست می‌آوریم. یک نمونه کد از Scheduler در اختیارتان قرار می‌گیرد و با استفاده از آن (یا در صورت دلخواه هر روشی که می‌توانید) باید این ترتیب اجرا عملیات‌ها با توجه به تعداد تکرار آنها و دوره هر یک به دست آورید.

```
configuration fib_demo;

  data a = 1;
  data b = 1;
  data n = 1;

  repeat (4) period=10 n = n + 1;
  repeat (4) period=10 a = a + b;
  repeat (4) period=10 b = a;

  repeat (2) period=20 display({n, a, b});
endconfiguration
```

همچنین در پیاده‌سازی خود موارد زیر را بهتر است رعایت کنید :

- برای متغیر boolean، اگر مقدار آن true باشد 1 و اگر مقدار آن false باشد 0 را در stack اضافه کنید.
- برای اضافه کردن مقادیر primitive به stack، از دستور ldc استفاده کنید.
- طول stack و locals را در متدها ۱۲۸ قرار دهید.
- برای انجام محاسبات روی Integer، مقادیر باید از نوع primitive یعنی int باشند.
- در اضافه کردن command ها به \n ها دقت داشته باشد تا در فایل jasmin ایجاد شده command ها پشت سر هم نباشند. همچنین هر command ای که اضافه می‌کنید به طور دقیق بررسی کنید که چه آرگومان‌هایی لازم دارد و چه مقداری را باز می‌گرداند؛ زیرا اگر اشتباهی رخ دهد debug کردن آن در فایل‌های jasmin کار دشواری است.

موارد پیاده‌سازی

پس از آنکه Scheduler که ترتیب اجرا عملیات را مشخص کرد،به ما می‌دهد به مرحله Code generation می‌رسیم. برای این قسمت نیز نمونه کدی در اختیار شما قرار می‌گیرد که شامل بدنه اصلی Code generation است و با تکمیل آن می‌توانید bytecode های مربوطه را تولید کنید.

پیاده‌سازی دستوراتی مانند موارد زیر از اهداف پروژه است و برای هر یک نمره‌ای در نظر گرفته شده است. پس لازم است تمامی آنها در کد شما پوشش داده شود:

- repeat ها (شامل assignment و یا display)
- component و انواع آن
- ساختار های expand (امتیازی)
- component با مقدار دیفالت و پارامتر (امتیازی)

در این قسمت تعدادی از کلاس‌ها و نحوه ایجاد کد برای آنها به صورت سطح بالا توضیح داده شده است که می‌توانید در پیاده‌سازی خود از آنها استفاده کنید.

تابع slotOf :

این تابع slot متغیرها را برمیگرداند. توجه کنید که slot صفر به صورت پیش‌فرض برای خود کلاس اصلی برنامه است و بعد از آن باید به ترتیب آرگومان‌های تابع اضافه شوند. برای دریافت یا ایجاد slot یک متغیر، کافیست تابع slotOf(varName i) صدا زده شود که در صورت وجود نداشتن، آن متغیر به slots اضافه می‌شود و slot مربوط به آن داده شود. همچنین در ابتدای هر تابع slots خالی می‌شود.

دستورات شرطی :

دستورات مورد نیاز برای دستور شرطی را اضافه کنید. دقت کنید که در این بخش label های مناسب را تولید کرده و در ادامه استفاده کنید.

حلقه‌های تکرار :

ابتدا label های مورد نظر را اضافه کنید. سپس statement های آن را visit کنید

BinaryExpression:

ابتدا عملوند سمت چپ و سپس عملوند سمت راست visit شوند و مقادیر آنها روی stack قرار بگیرند. سپس عملگر مورد نظر اعمال شود.

UnaryExpression:

ابتدا عملوند آن visit شود و مقدار آن روی stack قرار بگیرد، سپس عملگر مورد نظر اعمال شود.

Identifier :

از slot متناسب با آن identifier باید با استفاده از aload مقدار آن load شود.

Component ها :

دقت کنید برای هر یک از آنها باید فایل jasmin جدا ساخته شود.

دقت داشته باشید توصیه می‌شود برای درک بهتر ساختار آن و نحوه پیاده‌سازی آن به نمونه کد SimpleLang در ایلرن مراجعه کنید. جهت تولید فایل‌های class. نهایی از شما انتظار نمی‌رود که فایل باینری را مستقیما تولید کنید. برای این کار می‌توانید از اسمبلر jasmin استفاده کنید.

دستورات کاربردی jasmin

اضافه کردن به ArrayList

```
invokevirtual java/util/ArrayList/add(Ljava/lang/Object;)Z
```

گرفتن سایز ArrayList

```
invokevirtual java/util/ArrayList/size(I)
```

تبدیل (cast) یک Object به یک کلاس A

```
checkcast A
```

دستورات تبدیل و اجرای کدها

کامپایل کردن فایل java. به فایل class. :

```
javac -g *.java
```

اجرای فایل‌های class. (فایل Main.class باید اجرا شود):

```
java Main
```

تبدیل فایل jasmin bytecode(.j) به فایل class. :

```
java -jar jasmin.jar *.j
```

تبدیل فایل class. به java bytecode که خروجی هم در ترمینال نمایش داده شود:

```
javap -c -l A
```

تبدیل فایل class. به jasmin bytecode که خروجی هم در ترمینال نمایش داده شود:

```
java -jar classFileAnalyzer.jar A.class
```

نکات مهم

- **در صورت کشف هرگونه تقلب، نمره صفر لحاظ می‌شود.**
- **در صورت هرگونه سوال یا ابهام با طراحان پروژه از طریق ایمیل در ارتباط باشید.**
- **لینک ویدیوهای آشنایی با کدهای داده شده و** scheduler **داده شده است. توصیه میشود مشاهده کنید. به دلیل سرعت اینترنت ویدیو به دو قسمت تقسیم شده و در دو لینک زیر آپلود شده است.**

  https://my.files.ir/drive/s/2L11wLZPo8ry4eIPpVGU6s7sETHKPT

  https://my.files.ir/drive/s/5SQaFI5vEBvHlU4e1eYmqG3eCW7m09

- **دقت کنید که خروجی‌ها به صورت خودکار تست می‌شوند؛ پس نحوه چاپ خروجی باید عینا مطابق موارد ذکر شده در تست‌ها باشد. پیشنهاد می‌شود قبل از پیاده‌سازی،** **samples** **را بررسی کنید.**